6. What is compression?

Now let's understand compression.

Suppose your folder contains 100 MB of data.

You want to reduce its size to make it easier to transfer.

Compression algorithms encode the data more efficiently.

There are different compression formats in Linux.

Format	Purpose
`.tar`	Archive without compression
`.gz`	Gzip-compressed file
`.tar.gz`	Gzip-compressed tar archive
`.bz2`	Bzip2-compressed file
`.xz`	XZ-compressed file
`.zip`	Archive with compression

Important distinction:

- `.tar` combines files but does not compress them.
- `.tar.gz` combines files and compresses the resulting archive.

7. Gzip compression

`gzip` is used to compress files.

Syntax

```bash
gzip filename
```

Example:

```bash
gzip notes.txt
```

Now:

```
notes.txt.gz
```

will be created, and the original `notes.txt` will normally be replaced by its compressed version.

How to decompress?

```bash
gunzip notes.txt.gz
```

It restores:

```
notes.txt
```

Useful gzip commands

Command	Purpose
`gzip file.txt`	Compress a file
`gunzip file.txt.gz`	Decompress a file
`gzip -k file.txt`	Keep original while compressing
`gzip -d file.txt.gz`	Decompress
`gzip -l file.txt.gz`	Show compression information

Important: Use `gzip -k` if you want to preserve your original file.

8. Create a .tar.gz archive

This is one of the most important commands in Linux.

Suppose you want to archive and compress three files together.

Use:

```bash
tar -czvf backup.tar.gz notes.txt 1.txt akankshya.txt
```

Breaking it down:

Option	Meaning
`c`	Create
`z`	Gzip compression
`v`	Verbose
`f`	Filename

So:

```bash
tar -czvf backup.tar.gz *
```

will archive and gzip-compress the non-hidden entries in your current directory.

Be careful: this may also include an archive you created earlier if it is in that directory.

How to extract `.tar.gz` ?

```bash
tar -xzvf backup.tar.gz
```

Here `x` means extract.

Extract to another directory

```bash
mkdir backup-files
tar -xzvf backup.tar.gz -C backup-files
```

9. ZIP and UNZIP commands

You will also encounter `.zip` files frequently.

Unlike tar, ZIP supports both archiving and compression.

Install zip and unzip

If they aren't already installed:

```bash
sudo apt update
sudo apt install zip unzip
```

Create a ZIP archive

```bash
zip myfiles.zip notes.txt 1.txt akankshya.txt
```

ZIP an entire directory

```bash
zip -r myfolder.zip my-folder
```

Here `-r` means recursive, so it includes files inside subdirectories too.

View ZIP contents

```bash
unzip -l myfiles.zip
```

Extract ZIP

Bash

```
unzip myfiles.zip
```

Extract into a particular directory

Bash

```
mkdir zip-extracted
unzip myfiles.zip -d zip-extracted
```

10. Bzip2 and XZ compression

There are two more compression tools worth knowing.

Bzip2

Compress:

```bash
bzip2 notes.txt
```

Decompress:

```bash
bunzip2 notes.txt.bz2
```

Keep original:

```bash
bzip2 -k notes.txt
```

XZ

Compress:

```bash
xz notes.txt
```

Decompress:

```bash
unxz notes.txt.xz
```

Keep original:

```bash
xz -k notes.txt
```

Generally, XZ can achieve a smaller file size than gzip, but compression may take longer.

Write this table in your notebook.

Task	Command
Create TAR	`tar -cvf a.tar files`
Extract TAR	`tar -xvf a.tar`
List TAR contents	`tar -tvf a.tar`
Create TAR.GZ	`tar -czvf a.tar.gz files`
Extract TAR.GZ	`tar -xzvf a.tar.gz`
Compress with gzip	`gzip file`
Decompress gzip	`gunzip file.gz`
Create ZIP	`zip a.zip files`
ZIP directory	`zip -r a.zip folder`
Extract ZIP	`unzip a.zip`
List ZIP contents	`unzip -l a.zip`
Compress with bzip2	`bzip2 file`
Decompress bzip2	`bunzip2 file.bz2`
Compress with xz	`xz file`
Decompress xz	`unxz file.xz`

How Cron works



Cron Scheduler

Checks scheduled jobs every minute

↓

Scheduled time arrives

Example: Every day at 10 PM

↓



Command or script executes

Example: Run backup.sh

2. What is crontab?

`crontab` stands for **Cron Table**.

It is a file that contains the scheduled tasks for a user.

We use the `crontab` command to manage those tasks.

Important commands

Command	Purpose
`crontab -l`	List scheduled jobs
`crontab -e`	Edit scheduled jobs
`crontab -r`	Remove all scheduled jobs for the current user
`crontab -u username -l`	List another user's jobs (with sufficient privileges)
`crontab -u username -e`	Edit another user's jobs (with sufficient privileges)

Important: Be careful with `crontab -r`. It removes all your current user's cron jobs.

3. How to check your Cron jobs

Open your WSL Ubuntu terminal.

Type:

```bash
crontab -l
```

This displays your scheduled jobs.

If you haven't created any, you may see:

```
no crontab for akankshya
```

That is completely normal!

4. How to create your first Cron Job

Let's schedule a simple task.

Suppose you want Linux to display a message in a file every minute.

First, open the crontab editor:

```bash
crontab -e
```

If asked to choose an editor, select Nano if you're a beginner.

At the bottom, add:

```bash
* * * * * echo "Hello Akankshya" >> ~/cron-log.txt
```

Save and exit.

In Nano:

- `Ctrl + O` → Save.
- Enter → Confirm filename.
- `Ctrl + X` → Exit.

Now wait a minute and check:

Now wait a minute and check:

```bash
cat ~/cron-log.txt
```

You should see:

```
Hello Akankshya
```

Every minute, another line should be added.

5. Understanding Cron Job syntax

This is the MOST important part of Cron Jobs.

Every Cron Job follows this format:

```bash
* * * * * command
```

There are **five time fields** before the command.

The five Cron fields

Minute	Hour	Day	Month	Weekday
*	*	*	*	*
0–59	0–23	1–31	1–12	0–7

```plain text
* * * * * command
| | | | |
| | | | └── Day of week
| | | └──── Month
| | └────── Day of month
| └──────── Hour
└────────── Minute
```

Let's understand each field.

Field	Allowed values	Meaning
Minute	0–59	Which minute?
Hour	0–23	Which hour?
Day	1–31	Which date?
Month	1–12	Which month?
Weekday	0–7	Which day of the week?

For weekday:

- 0 = Sunday
- 1 = Monday
- 2 = Tuesday
- 3 = Wednesday
- 4 = Thursday
- 5 = Friday
- 6 = Saturday
- 7 = Sunday (also accepted)

6. Understanding the special symbols

Cron uses some special symbols.

Symbol	Meaning
`*`	Every value
`,`	Multiple specific values
`-`	Range of values
`/`	Step or interval

Let's understand with examples.

A. Asterisk `*`

```bash
* * * * * command
```

Meaning: Execute every minute.

B. Comma `,`

```bash
0 9,18 * * * command
```

Meaning: Execute at 9 AM and 6 PM every day.

C. Hyphen `-`

```bash
0 9-17 * * * command
```

Meaning: Execute every hour from 9 AM to 5 PM, at minute 0.

D. Slash `/`

```bash
*/5 * * * * command
```

Meaning: Execute every 5 minutes.

7. Important Cron Job examples

These are useful examples to write in your notebook.

Cron expression	Execution time
`* * * * *`	Every minute
`*/5 * * * *`	Every 5 minutes
`*/10 * * * *`	Every 10 minutes
`0 * * * *`	Every hour
`0 8 * * *`	Every day at 8 AM
`0 22 * * *`	Every day at 10 PM
`30 9 * * *`	Every day at 9:30 AM
`0 0 * * *`	Every midnight
`0 8 * * 1`	Every Monday at 8 AM
`0 8 * * 0`	Every Sunday at 8 AM
`0 9 1 * *`	First day of every month at 9 AM
`0 8 * * 1-5`	Weekdays at 8 AM